

ANSWER SHEET DAC 2025

PRELIMINARY ROUND



**Data Analysis
Competition 2025**

Team Name : epilogkuskus

Himpunan Mahasiswa Statistika ITS (HIMASTA-ITS)

Gedung H Lantai III, Jl. Arief Rahman Hakim
Kampus ITS Sukolilo Surabaya

Email: pekanrayastatistika2025@gmail.com

Contact Person:

Hita (+62 857-1207-7572)

Ali (+62 812-3590-7687)

CHAPTER I: Introduction

As Large Language Models (LLMs) such as GPT, Claude, Gemini, and others continue to grow in popularity, the usage of it has become vary. It has become integrated into a wide range of purposes, from conversational assistants to productivity tools. With this range of applications, user reviews has become an essential metric in determining the quality of an LLM. Unfortunately, in many cases, the reviews weren't in the form of binary choices such as “good app” or “bad app”, but more in a deeper sentiment signals that represent trust, satisfaction, frustration, or even skepticism toward the technology. For LLMs developers, understanding these reviews and categorizing the goods and bads is important in creating a fully functional and human-centered product. Via these reviews, LLMs developers can detect pain points, identify strengths of LLM being developed, and can serve as a competitive benchmarking across the LLM ecosystem that are usually not visible through numerical features or usage metrics.

However, extracting such insight is far from a trivial task. High variation of user reviews and the sheer volume of it makes manual analysis infeasible and can be time-consuming. As mentioned earlier, in real-data, a review itself can contain positive and negative sentiment simultaneously. Furthermore, the review can also contain an abbreviation or *slang* word that the model can't capture immediately using traditional machine learning. A class imbalance is also another problem that always occurs in sentiment analysis. Users tend to leave positive feedback compared to neutral or negative ones most likely to avoid any hassle, unless it's frustrating enough to the point that it requires a complaint.

To address these challenges, this paper will focus on developing a machine learning (ML) model to automate the task of sentiment analysis toward customer review across multiple LLMs (e.g GPT, Claude, Gemini, Grok, Perplexity, and DeepSeek). The system that will be developed was designed to handle imbalance data and in the end, will be able to classify review sentiment into three categories consisting of 0 : negative, 1 : neutral/mixed, and 2 : positive. Hopefully, through this paper, the task of sentiment analysis for analyzing reviews can be more accessible to the general public and can help LLMs developers build a more reliable and human-centered product. Also, by building such a model, this work aims not only to advance sentiment analysis in the context of LLM feedback but also to highlight the importance of fairness, robustness, and generalizability in real-world NLP deployment.

CHAPTER II: Theoretical Framework

2.1. Natural Language Processing

The study of how computers and human language interact is the focus of the Artificial Intelligence (AI) subfield of Natural Language Processing (NLP). Machines can now comprehend, evaluate, and produce natural language in the form of speech or text thanks to natural language processing (NLP). NLP is frequently used in text classification, machine translation, and sentiment analysis. The ambiguity of language, the diversity of linguistic structures, and the difficulty of accurately modeling context are some of the main obstacles in natural language processing (NLP) [2].

2.2. Text Representation and Model Development

Text representation is a fundamental step in natural language processing (NLP) that transforms raw text into numerical vectors that can be processed by machine learning algorithms. Early representations such as Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) provided a way to quantify word occurrences, but they were limited in capturing semantic meaning and contextual relationships. Distributed representations such as Word2Vec [5] and GloVe [6] improved upon these methods by embedding words in dense vector spaces where semantic similarity is reflected in geometric proximity. However, these static embeddings cannot adapt to different contexts, assigning the same vector to a word regardless of its surrounding text.

To overcome these limitations, contextual embeddings were introduced, where word meaning is represented based on the surrounding text. Models such as BERT and its extensions brought significant improvements by learning contextualized word embeddings that adapt dynamically depending on usage. A key development in this field is XLM-RoBERTa [3], a multilingual language model trained on 2.5 terabytes of CommonCrawl data across 100 languages.

2.3. Transformer and XLM-RoBERTa

Vaswani et al. (2017) developed the Transformer architecture, which replaced traditional sequence models such as RNNs and LSTMs with a self-attention mechanism. Unlike recurrent models, transformers can capture long-range dependencies and process all tokens in a sequence at the same time. Transformers' ability to model and work quickly made them the most important part of cutting-edge NLP systems. The central component of the Transformer is the *scaled dot-product attention*, which computes attention weights as

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where Q represents the query matrix, K the key matrix, V the value matrix, and d_k the dimension of the key vectors. This mechanism allows the model to weigh the importance of different tokens relative to each other in a sequence, thereby capturing contextual relationships effectively.

Building upon this foundation, XLM-RoBERTa [1] extends the RoBERTa model to a multilingual setting, having been pre-trained on 2.5TB of filtered CommonCrawl data across 100 languages. Unlike earlier cross-lingual models that rely on parallel corpora, XLM-RoBERTa employs a masked language modeling (MLM) objective, where random tokens are masked and the model learns to predict them based on their context. This approach leverages dynamic masking and large-scale training to produce powerful multilingual representations. As a result, XLM-RoBERTa has demonstrated state-of-the-art performance on a wide range of cross-lingual benchmarks, such as XNLI (Cross-lingual Natural Language Inference) and MLQA (Multilingual Question Answering).

The integration of Transformers and XLM-RoBERTa in research provides a robust theoretical foundation for addressing classification tasks in multilingual datasets. The Transformer's attention mechanism ensures efficient context modeling, while XLM-RoBERTa's multilingual training facilitates cross-lingual transfer and generalization. This synergy is particularly valuable for tasks such as sentiment analysis, stance detection, and other text classification problems where data distribution is imbalanced across languages. By leveraging these models, researchers can achieve higher accuracy, stability, and generalization, making them indispensable tools in modern NLP research.

2.4. Regularization and Optimization in Deep Learning

In deep learning, regularization techniques are crucial to prevent models from overfitting to training data and improve their ability to generalize to unseen examples. One of the most widely used approaches is **L2 regularization** (also called weight decay), which penalizes large weight values by adding a term to the loss function described in the following equation.

$$L_{reg}(\theta) = L(\theta) + \lambda \sum_{i=1}^n \theta_i^2,$$

where $L(\theta)$ is the original loss, θ_i are the model parameters, and λ is a hyperparameter controlling the strength of the penalty. Another common method is dropout, which randomly sets a fraction p of the neurons to zero

during training, effectively preventing co-adaptation of hidden units and encouraging redundancy in learned representations [7].

On the other hand, optimization algorithms are designed to minimize the loss function efficiently. The most widely adopted optimizer in modern NLP is **Adam** (Adaptive Moment Estimation), which combines momentum and adaptive learning rates per parameter. Its parameter update rule is as follow

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

where α is the learning rate, $\hat{m}_t$ is the bias-corrected first moment estimate (mean of gradients), and $\hat{v}_t$ is the bias-corrected second moment estimate (uncentered variance of gradients) [8]. To further improve training stability, **learning rate scheduling** (e.g., cosine annealing or linear decay) is often used, which gradually reduces the learning rate over epochs to allow fine convergence [3]. Together, regularization and optimization strategies ensure that deep learning models, such as transformer-based architectures, converge effectively while maintaining strong generalization performance on real-world, imbalanced datasets.

2.5. Model Evaluation

Precision, Recall, and F1-score are widely used metrics to evaluate the performance of text classification systems. Precision measures the proportion of correctly predicted positive samples out of all predicted positive samples:

$$Precision = \frac{TP}{TP + FP}.$$

On the other hand, Recall measures the proportion of correctly predicted positive samples out of all actual positive samples:

$$Recall = \frac{TP}{TP + FN}.$$

The F1-score is the harmonic mean of Precision and Recall, balancing the trade-off between them:

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}.$$

Macro-F1 calculates the F1-score independently for each class and then averages them, treating all classes equally regardless of their frequency. In contrast, Weighted-F1 multiplies each class's F1-score by the proportion of samples in that class before averaging, giving more influence to frequent classes. For example, in a sentiment dataset with 80% neutral, 15% positive,

and 5% negative samples, Macro-F1 would treat each class equally, potentially exaggerating the impact of minority classes. Weighted-F1, however, accounts for class proportions, providing a more realistic measure of overall performance that reflects the actual distribution of the data. Monitoring training loss and validation loss during model training is also essential to track convergence and detect potential overfitting, ensuring that the model generalizes well to unseen data [3].

CHAPTER III: Analytical Steps

3.1. Analytical Steps Environment

The analysis was being conducted entirely on the Kaggle Platform using Python 3.11.13. Kaggle provides a cloud-based environment equipped with CPUs, GPUs, and TPUs, which facilitates efficient model training and experimentation. The experiments were also conducted on GPU accelerators (NVIDIA Tesla P100, 16GB memory) enabled. This hardware setup significantly reduced training time for deep learning models compared to CPU-only execution.

The following libraries and frameworks were utilized throughout the workflow.

1. Data Processing & Utilities: `pandas`, `numpy`, `collections`, `re`, `warnings`.
2. Machine Learning & Evaluation: `scikit-learn` (train-test splitting, stratified cross-validation, class weights, logistic regression, TF-IDF vectorization), `evaluate`.
3. Deep Learning Framework: `torch`, `torch.nn`, `torch.nn.functional`.
4. Natural Language Processing (NLP): Hugging Face `transformers` (AutoTokenizer, AutoModelForSequenceClassification, Trainer API, cosine learning rate scheduler, early stopping) and `datasets` for dataset management.
5. Other Tools: `emoji` for text preprocessing involving emoji normalization.

Random seeds were fixed across `numpy` and `torch` to improve reproducibility. The GPU accelerator significantly reduced training time, particularly for transformer-based models. Data was accessed directly from Kaggle's address `/kaggle/input/data-analysis-competition-2025` directory, ensuring seamless integration with competition datasets.

To improve model robustness and generalization, several advanced hyperparameter strategies were applied:

- **Loss Functions & Regularization:**

- Focal Loss was enabled (`USE_FOCAL_LOSS = True`) to handle class imbalance by giving higher weight to hard-to-classify samples.
- Label smoothing (`0.1`) was applied to reduce overfitting by softening class labels.
- **Dropout Regularization:**
 - Standard dropout: `0.3`
 - Attention dropout: `0.2`
 - Hidden layer dropout: `0.3`. These were introduced to prevent overfitting in both attention and hidden layers.
- **Training Dynamics:**
 - Warmup ratio set to `0.1` for gradual learning rate ramp-up.
 - Weight decay of `0.01` was used for L2 regularization.
 - Maximum sequence length (`MAX_LENGTH`) increased to 256 tokens to capture broader context.
 - Gradient accumulation (2) allowed effective larger batch sizes within GPU memory limits.
- **Model Validation & Ensembling:**
 - 3-fold cross-validation (`N_FOLDS = 3`) was adopted to ensure robustness.
 - Ensembling (`USE_ENSEMBLE = True`) across folds was performed to improve stability and reduce variance of predictions.

3.2. Data Collection and Preprocessing

The dataset was collected from six LLMs: GPT, Claude, DeepSeek, Gemini, Grok, and Perplexity. Each model provided one training and one testing dataset. Training data contained: Comment, At, AppVersion, Sentiment. Testing data contained: CommentId, Comment, At, AppVersion. Overall, the dataset represents user feedback and experiences with each LLM. Comments appear in diverse forms, including plain text, emojis, links, or combinations of these, requiring careful preprocessing to handle noise and preserve semantic meaning.

The code begins with a data loading procedure that combines training and testing datasets from six different LLMs. Each dataset is read, assigned a unique CommentId, and tagged with its corresponding source model to preserve origin information. After concatenation, unnecessary timestamp columns are dropped, resulting in unified training and testing sets. Following this, an advanced preprocessing pipeline is applied to standardize and clean user comments. This process includes normalizing slang expressions in both Indonesian and English, converting emojis into descriptive tokens, compressing repeated characters, and replacing excessive punctuation patterns (e.g., `!!!` to `EXCITED`, `???` to `CONFUSED`). Special cases such as URLs, hashtags, mentions, numbers, and all-caps tokens are also replaced

with standardized markers, while a negation-handling mechanism ensures that words following negation terms are tagged appropriately to retain sentiment meaning.

To further improve dataset reliability, a mislabeled data detection and correction stage is introduced. This stage employs a baseline TF-IDF representation with Logistic Regression to predict sentiment probabilities for each sample. Samples that show a strong disagreement between the model's prediction and the original label, or that contain conflicting sentiment keywords, are flagged as potentially mislabeled. Depending on the chosen strategy, these samples can be dropped, reweighted, or relabeled. In this case, the relabeling strategy is applied, automatically correcting labels to align with the model's confident predictions. This combination of structured loading, robust preprocessing, and label quality control ensures that the training data is both consistent and reliable for downstream model development.

3.3. Data Augmentation

In this step, data augmentation is applied with the main goal of addressing the issue of class imbalance in the training dataset. Class imbalance often occurs when certain sentiment categories (e.g., positive, neutral, or negative) contain significantly fewer samples compared to the majority class. This imbalance can bias the model toward predicting the majority class more frequently, leading to weaker performance on underrepresented classes. To mitigate this issue, the function `augment_minority_classes` is introduced. It first inspects the label distribution in the `Sentiment` column, then selectively increases the sample size for minority classes whose frequency is less than half of the majority class (as defined by the parameter `aug_factor = 0.5`).

The augmentation strategy used in this implementation is a simple form of paraphrasing through word shuffling. The process starts by randomly selecting up to 100 samples from each minority class. For each selected comment, the sentence is split into individual words, and then the order of some words is shuffled to create slightly altered sentence structures. Importantly, negation words are preserved in their original positions to ensure that the overall sentiment meaning of the comment remains consistent. The resulting augmented comment is then assigned a modified `CommentId` with an `_aug` suffix, making it distinguishable from the original data.

Finally, the augmented samples are concatenated back into the training set, effectively expanding the representation of minority classes. While this augmentation technique is relatively simple, it still helps introduce variability in the textual data and prevents the model from overfitting to limited sentence patterns. As a result, the dataset becomes more balanced across sentiment

categories, which is expected to improve the model's generalization performance when dealing with real-world inputs.

3.4. Loss Function

To further improve model robustness and address class imbalance, we integrated advanced loss functions into the training process. Specifically, we implemented Focal Loss and a Combined Loss that merges Focal Loss with Cross-Entropy Loss. Focal Loss is designed to handle cases where some classes dominate the dataset, making the model biased toward the majority class. It works by down-weighting “easy” examples that the model can classify confidently and focusing more on “hard” or misclassified samples. The loss for a single prediction can be expressed as:

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \cdot \log(p_t)$$

where p_t is the predicted probability for the correct class, α is a balancing factor for class weights, and γ (gamma) controls the degree to which difficult examples are emphasized. We also applied label smoothing, which prevents the model from becoming overly confident in its predictions by slightly softening the target labels. This improves generalization and reduces overfitting. In addition, we implemented a Combined Loss function that leverages both Focal Loss and Cross-Entropy Loss. The idea is to benefit from the stability of Cross-Entropy while retaining the ability of Focal Loss to focus on minority and hard-to-classify instances. The final combined objective is defined as:

$$L_{combined} = \lambda \cdot L_{local} + (1 - \lambda) \cdot L_{CE}$$

where λ is a weighting parameter (e.g., 0.7 for Focal Loss and 0.3 for Cross-Entropy). This hybrid approach ensures that the model not only learns effectively from balanced classes but also adapts well to nuanced and challenging samples, ultimately improving overall classification performance.

3.5. Robust Focal Loss

To ensure compatibility across different models and improve stability during training, we implemented a Robust Focal Loss. This function extends the standard Focal Loss by incorporating data type safety, label smoothing, and improved numerical stability. This is essential because the basic version sometimes fails when applied to different architectures due to strict input requirements. Our Robust Focal Loss introduces the following improvements:

1. Data type consistency – It ensures logits and targets are converted into the correct format (float for inputs and long for targets), avoiding runtime errors across various model outputs.

2. Label smoothing support – Instead of relying solely on hard one-hot labels, we soften the targets by distributing a small probability mass across all classes. This prevents overconfidence and enhances generalization. When label smoothing is enabled, the loss is computed via KL divergence between the predicted distribution and the smoothed targets.
3. Flexible reduction modes – Supports "mean", "sum", or "none", allowing better control over how the loss is aggregated.

3.6. Define Training Pipeline

This section implements an advanced fine-tuning pipeline for transformer-based models using the Hugging Face `Trainer` API, but it extends the standard setup with several robustness and optimization features. The core innovation lies in the `AdvancedTrainer` class, which overrides the default `compute_loss` method. Instead of relying solely on the standard cross-entropy loss, it integrates a custom `CombinedLoss` function (built on top of `RobustFocalLoss`) that combines focal loss, label smoothing, and class weighting. This design mitigates issues such as class imbalance, where minority classes might otherwise be underrepresented, and noisy labels, where label smoothing can soften overconfident predictions. The focal loss term specifically reweights difficult-to-classify samples, preventing the model from being dominated by easy examples.

The `create_model_and_tokenizer` function standardizes model initialization across architectures. It loads a model and tokenizer from Hugging Face's hub and applies dropout regularization parameters (`hidden_dropout_prob`, `attention_probs_dropout_prob`, and `classifier_dropout`) if supported by the model's config. This increases model robustness by reducing overfitting.

The `train_single_model` function orchestrates the full training pipeline. It first initializes the model/tokenizer and sets up **training arguments** tailored for competitive performance:

- **Cosine learning rate scheduling** with a warmup ratio to stabilize early training.
- **Gradient accumulation** to simulate larger batch sizes without exceeding GPU memory.
- **FP16 mixed precision training** for computational efficiency.
- **Weight decay** for additional regularization.
- **Early stopping** to prevent overfitting and unnecessary computation.

For evaluation, the code leverages the `evaluate` library to compute both **macro-F1** (which treats all classes equally, important in imbalanced settings) and **weighted-F1** (which accounts for class distribution). This provides a more comprehensive assessment of performance compared to accuracy.

Finally, the trainer supports **checkpoint resumption**, allowing training to continue seamlessly if interrupted, and **best model loading** based on validation F1. Together, these components form a **robust, efficient, and generalizable training pipeline** that is well-suited for research and competitive machine learning tasks where robustness against imbalance and overfitting is critical.

3.7. Cross Validation and Ensemble

This section will implement a cross-validation ensemble pipeline for text classification. It begins with a StratifiedKFold split to preserve sentiment class balance across folds, preventing bias in imbalanced datasets. For each fold, training and validation subsets are created from `train_data`, while `test_data` stays constant. Hugging Face Datasets are built with the Sentiment column renamed to labels for compatibility with the Trainer API.

Each model in `MODELS` is trained separately using its own tokenizer to ensure alignment with its pretraining vocabulary. Data is tokenized and converted into PyTorch tensors (`input_ids`, `attention_mask`, and labels for training/validation, excluding labels for test). Training is handled by `train_single_model`, and after training, raw logits are generated as predictions. To prevent GPU memory overflow, trained objects are deleted and `torch.cuda.empty_cache()` is called after each run.

Within each fold, model outputs are averaged (soft-voting ensemble) to produce stable predictions, and validation scores are tracked. Finally, the function returns: `all_predictions` (aggregated ensemble predictions across folds) and `cv_scores` (averaged cross-validation scores, reflecting generalization ability).

3.8. Main Execution

This section will set up and run a single-model training and prediction pipeline without cross-validation. It first computes class weights based on the distribution of sentiment labels, ensuring that the loss function compensates for imbalanced classes. The dataset is then split into training (85%) and validation (15%) subsets using stratified sampling to preserve class proportions.

Next, the best-performing model (selected from `MODELS`) is chosen, and the datasets are prepared in Hugging Face Dataset format with the Sentiment column renamed to labels. A tokenizer specific to the chosen model is applied, converting text comments into tokenized inputs (`input_ids`, `attention_mask`) with truncation and padding. The data is formatted into PyTorch tensors, ready for supervised learning.

The training is carried out via the `train_single_model` function, which returns both the trained trainer object and validation score. Finally,

predictions are made on the test set, and the class with the highest probability (via `argmax`) is selected as the output label for each instance.

CHAPTER IV: Analysis of Results

4.1. Sentiment Classification Modeling

4.1.1. Data Preprocessing

In the preprocessing stage, user comments in the dataset are cleaned and converted into a structured format suitable for machine learning modeling. Some preprocessing techniques used include converting all text to lowercase, removing punctuation, numbers, and emojis, and removing stopwords that have no significant meaning. Next, we perform the tokenization stage. Tokenization is used to break the text into words, followed by lemmatization or stemming to return the words to their basic form. To demonstrate the results of this stage, several raw comments are compared with the processed versions, demonstrating how the text has become simpler and more standardized.

Table 4.1. Before vs after Preprocessing

Id	Raw Comment	Preprocessing comment
gpt_1	super bussniss idea 💡 for chat gpt birrlant ap	super bussniss idea light_bulb for chat gpt birrlant ap
gpt_20	dito na nga lang ako nkakapag rant nag ka limit pa ano ba naman buhay to oh 😞💔💔	dito na nga lang ako nkakapag rant nag ka limit pa ano ba naman buhay to oh pensive_face broken_heart broken_heart
gpt_26	There is no better program than this. I really, really loved it. ❤️❤️	there is no NEG_better NEG_program NEG_than this. i really, really loved it. red_heart red_heart
gpt_27	Good 👍	good thumbs_up

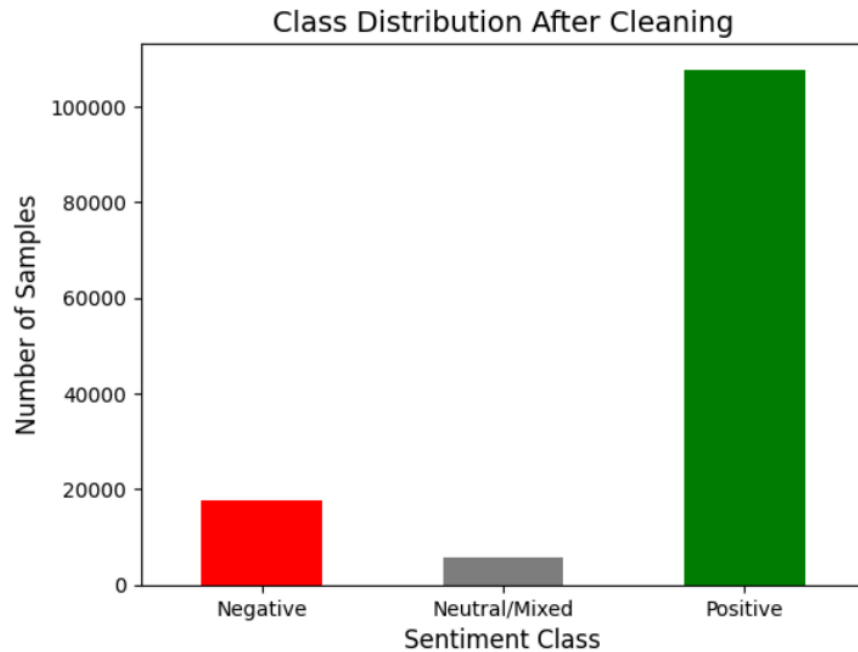
This stage is carried out to ensure that the input given to the model is more consistent and free from noise, thus improving model performance.

4.1.2. Data Quality & Distribution

After the data is cleaned, the quality and distribution of sentiment classes are checked. The dataset is examined to ensure a valid sample size and identify any mislabeled data. The sentiment distribution is visualized using a bar chart to determine whether the

dataset is balanced or class-imbalanced. This is important because class imbalance can impact model performance.

Picture 4.1. Sentiment distribution visualization



Based on the bar chart, there is a data imbalance, where the number of positive sentiments is much greater than negative and neutral sentiments, this can cause bias in the majority (positive) class and reduce model performance in the minority class.

4.1.3. Data Augmentation

To address the limitation of data, especially for minority classes, data augmentation was applied. Techniques such as synonym replacement or back translation were used to introduce variation in sentences without altering their meaning. As a result, the class distribution became more balanced.

Picture 4.2. Data Count Comparison Before and After Augmentation



4.1.4. Model Training & Optimization

After the data is ready, the sentiment classification model is trained using the BERT algorithm (XLM-RoBERTa model). This training process uses a train-validation split to measure performance on data the model did not encounter during training.

During the training process, hyperparameters such as learning rate, batch size, number of epochs, and the optimizer used are adjusted. This optimization is crucial for achieving optimal model performance. The results of each epoch are recorded, including the loss value and F1-score, and checkpoints are created to allow the model to continue training at the checkpoints.

Picture 4.3. hyperparameters set up

```
training_args = TrainingArguments(
    output_dir=f"{OUTPUT_DIR}/{model_name.replace('/', '_')}_fold_{fold_idx}",
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    greater_is_better=True,
    learning_rate=1e-5,
    lr_scheduler_type="cosine",
    warmup_ratio=WARMUP_RATIO,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
    num_train_epochs=4,
    weight_decay=WEIGHT_DECAY,
    logging_dir=f"{OUTPUT_DIR}/logs_{fold_idx}",
    logging_strategy="steps",
    logging_steps=50,
    report_to="none",
    disable_tqdm=False,
    seed=42 + fold_idx,
    fp16=True,
    dataloader_num_workers=2,
    remove_unused_columns=True,
    push_to_hub=False,
)
```

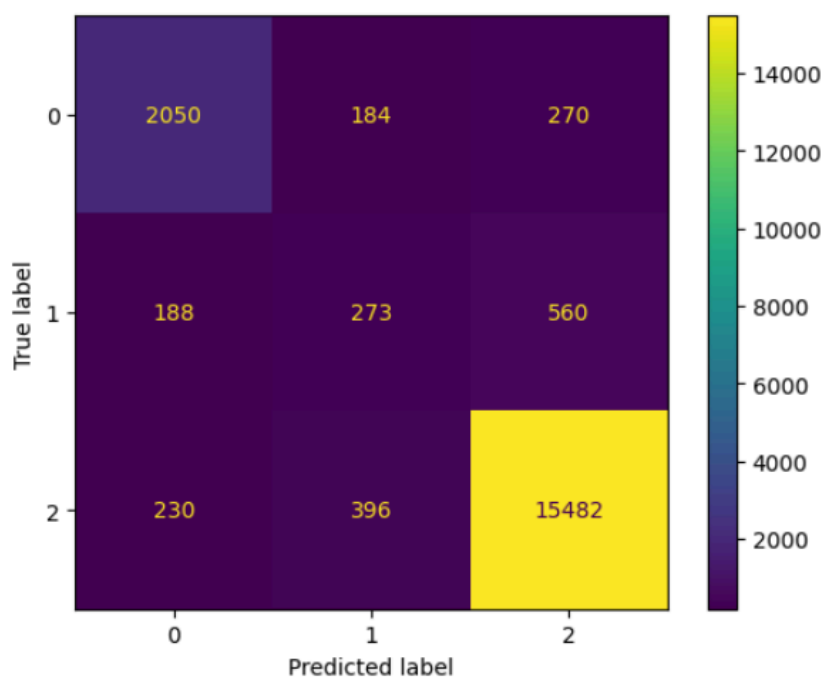
4.1.5. Model Evaluation

The Macro-F1 Score and Weighted-F1 Score metrics are used for model evaluation. Macro-F1 provides an overview of the average performance per epoch, while Weighted-F1 takes into account the proportion of data in each epoch. A confusion matrix is also displayed for a more detailed look at the level of prediction accuracy per class. The evaluation results indicate the model's ability to handle imbalanced data distributions.

Picture 4.4. Macro-F1 Score and Weighted-F1 Score

Epoch	Training Loss	Validation Loss	F1	F1 Weighted
1	0.337900	0.321315	0.679887	0.897485
2	0.259200	0.369892	0.686502	0.901429
3	0.294000	0.357911	0.690354	0.903932

Picture 4.5. Confusion Matrix

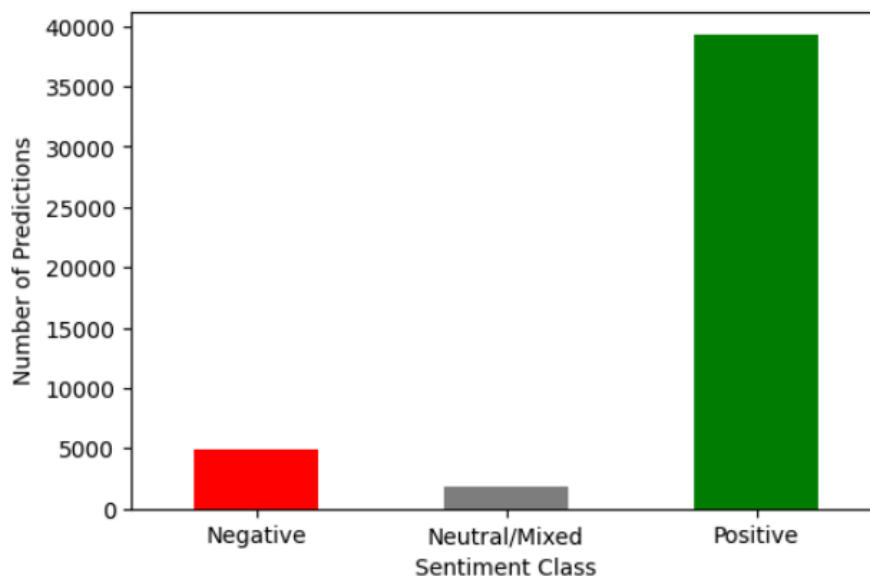


Based on the confusion matrix, it can be concluded that the model's performance in predicting sentiment from the minority class is still less good when compared to the model's performance in the majority class. This is inseparable from the imbalance of the dataset between the majority and minority classes.

4.1.6. Prediction Distribution on Test Data

After training and evaluation, the model was applied to the test set. The distribution of predictions was presented in tables and bar charts to show the proportion of each sentiment class. This analysis provided insight into whether the model was biased towards certain classes or produced relatively balanced predictions.

Picture 4.6. Prediction Distribution on Test Data

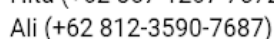


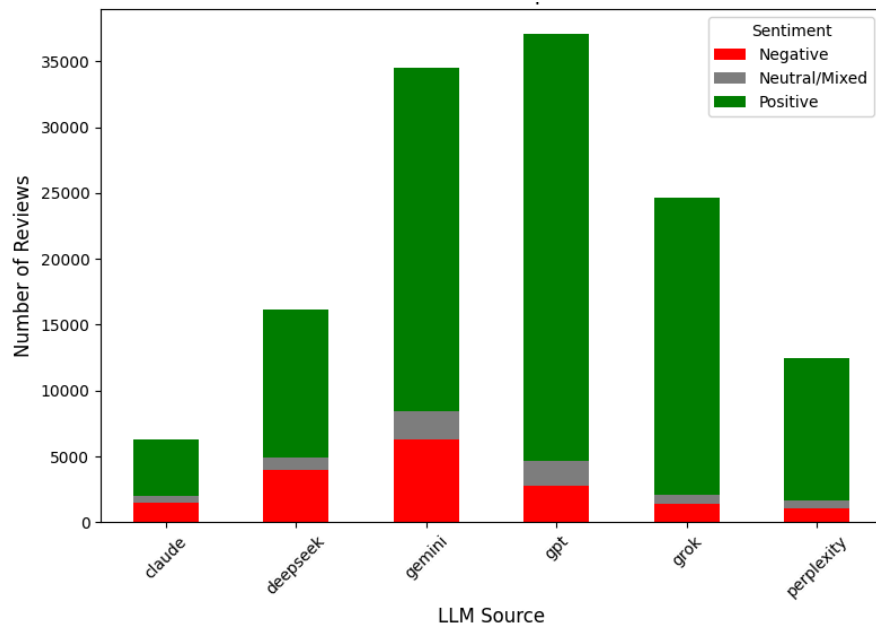
4.2. Insights from User Reviews

4.2.1. Analysis of Negative Reviews

To delve deeper, reviews with negative sentiment were specifically analyzed. The goal was to identify the most frequently occurring keywords or issues, thus providing insight into the causes of user dissatisfaction. The method used was frequency analysis, visualized using word clouds.

Picture 4.7. Negative word clouds before preprocessing





CHAPTER V: Conclusion and Recommendation

This study aims to test the effectiveness of the XLM-RoBERTa-base model in NLP to classify sentiment tendencies from given comments/arguments. The analysis results showed that the fine-tuned XLM-RoBERTa-based model performed quite well in sentiment classification, with an F1 score (macro) within the training interval of 0.67-0.69, indicating the model's ability to capture general trends. However, due to the unbalanced dataset, where there is a very large difference between positive, neutral and negative sentiments, this impacts the model's performance in classifying the minority class. Apart from that, there is some incorrect data labeling, so additional preprocessing needs to be done to re-label the incorrect labels.

The findings from this analysis indicate that positive comments tend to be more consistent, while negative and neutral comments are more diverse. This makes it difficult for the model to recognize patterns in the minority class. Therefore, imbalanced data handling strategies such as oversampling, class weight adjustment, or data augmentation become important to improve prediction accuracy. Furthermore, exploring keywords or language patterns frequently appearing in negative and neutral comments could provide additional insights useful for further model development.

Overall, XLM-RoBERTa-base proved effective in capturing general sentiment patterns, but performance on minority classes could still be improved. Practical recommendations from this analysis include implementing data balancing techniques, studying language patterns in comments, and optimizing model hyperparameters. This approach is expected to achieve more balanced and accurate sentiment predictions, maximizing insights from the data and benefiting decision-making in competitions and future research.

REFERENCES

- [1] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” in Proc. 58th Annual Meeting Assoc. Computational Linguistics (ACL), 2020, pp. 8440–8451.
- [2] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed., draft. Stanford, CA, USA: Stanford Univ., 2021.
- [3] S. Raschka and V. Mirjalili, Machine Learning with PyTorch and Scikit-Learn. Birmingham, U.K.: Packt Publishing, 2020.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.
- [5] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [6] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global vectors for word representation,” in Proc. Conf. Empirical Methods Natural Language Processing (EMNLP), 2014, pp. 1532–1543.
- [7] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Improving neural networks by preventing co-adaptation of feature detectors,” *arXiv preprint arXiv:1207.0580*, 2012.
- [8] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. Int. Conf. Learn. Representations (ICLR), 2015.



Data Analysis
Competition 2025

DATA ANALYSIS COMPETITION

INSTITUT TEKNOLOGI SEPULUH NOPEMBER



Himpunan Mahasiswa Statistika ITS (HIMASTA-ITS)

Gedung H Lantai III, Jl. Arief Rahman Hakim
Kampus ITS Sukolilo Surabaya

Email: pekanrayastatistika2025@gmail.com

Contact Person:

Hita (+62 857-1207-7572)

Ali (+62 812-3590-7687)